UNIDAD 3 – JAVASCRIPT NATIVO

DESARROLLO WEB EN ENTORNO CLIENTE

Indice

- Tipos de datos (avanzado).
- Objetos.
- Clases.
- Manejo de errores.

Tipos de datos.

- **Números**.
  - Tostring(base).

```
let num = 255;

alert( num.toString(16) );  // ff
alert( num.toString(2) );   // 11111111
```

  - Redondeo.

	Math.floor	Math.ceil	Math.round	Math.trunc
3.1	3	4	3	3
3.6	3	4	4	3
-1.1	-2	-1	-1	-1
-1.6	-2	-1	-2	-1

```
let num = 1.23456;

alert( Math.round(num * 100) / 100 ); // 1.23456 -> 123.456 -> 123 -> 1.23
```

Tipos de datos.

- **Números**.
  - parseInt, parseFloat.

```
1  alert( parseInt('100px') ); // 100
2  alert( parseFloat('12.5em') ); // 12.5
3
4  alert( parseInt('12.3') ); // 12, devuelve solo la parte entera
5  alert( parseFloat('12.3.4') ); // 12.3, el segundo punto detiene la lectura
```

  - Otras funciones matemáticas.
  - Math.random().
  - Math.max().
  - Math.pow().

Tipos de datos.

- **String**.
  - **\n:** salto de linea.

```
let guestList = 'Invitados:\n * Juan\n * Pedro\n * Maria';

alert(guestList); // lista de invitados en múltiples líneas,
```

Carácter	Descripción
`\n`	Nueva línea
`\r`	En Windows, los archivos de texto usan una combinación de dos caracteres `\r\n` para representar un corte de línea, mientras que en otros SO es simplemente '\n'. Esto es por razones históricas, la mayoría del software para Windows también reconoce '\n'.
`\'`, `\"`, `\``	Comillas
`\\`	Barra invertida
`\t`	Tabulación
`\b`, `\f`, `\v`	Retroceso, avance de formulario, tabulación vertical – Se mencionan para ser exhaustivos. Vienen de muy viejos tiempos y no se usan actualmente (puedes olvidarlos ya).

Tipos de datos.

- **String**.
  - Propiedad length.

```
alert(`Mi\n`.length); // 3
```

  - Accediento da caracteres.

```
let str = `Hola`;

// el primer carácter
alert( str[0] ); // H
alert( str.at(0) ); // H

// el último carácter
alert( str[str.length - 1] ); // a
alert( str.at(-1) );
```

  - Iterar sobre caracteres.

```
for (let char of 'Hola') {
  alert(char); // H,o,l,a (char se convierte en "H", luego "o", luego "l", etc.)
}
```

Tipos de datos.

- **String**.
  - toUpperCase, toLowerCase.

```
1  alert('Interfaz'.toUpperCase()); // INTERFAZ
2  alert('Interfaz'.toLowerCase()); // interfaz
```

  - Buscar subcadenas. indexOf, lastIndexOf

```
1  let str = 'Widget con id';
2
3  alert(str.indexOf('Widget')); // 0, ya que 'Widget' es encontrado al comienzo
4  alert(str.indexOf('widget')); // -1, no es encontrado, la búsqueda toma en cuenta mi
5
6  alert(str.indexOf('id')); // 1, "id" es encontrado en la posición 1 (..idget con id)
```

  - Includes, starsWith, endsWith.

```
1  alert('Widget con id'.includes('Widget')); // true
2
3  alert('Hola'.includes('Adiós')); // false
```

```
1  alert( "Widget".startsWith("Wid") ); // true, "Widget" comienza con "Wid"
2  alert( "Widget".endsWith("get") ); // true, "Widget" termina con "get"
```

Tipos de datos.

- **String**.
  - Obtener subcadena. Substring, substr, slice.

```
let str = "stringify";
alert( str.slice(0, 5) ); // 'strin',
alert( str.slice(0, 1) ); // 's', desc
```

```
let str = "stringify";
// comienza en la 4ta posición desde la derecha, finaliza en la 1era posición
alert( str.slice(-4, -1) ); // 'gif'
```

```
let str = "stringify";

// esto es lo mismo para substring
alert( str.substring(2, 6) ); // "ring"
alert( str.substring(6, 2) ); // "ring"

// ...pero no para slice:
alert( str.slice(2, 6) ); // "ring" (lo mismo)
alert( str.slice(6, 2) ); // "" (un string vacío)
```

```
let str = "stringify";
alert( str.substr(2, 4) ); // ring, desde la 2nda posición toma 4 caracteres
```

Tipos de datos.

- **String**.
  - Obtener código para el carácter y viceversa. str.codePointAt(pos).

```
1  // mayúsculas y minúsculas tienen códigos diferentes
2  alert( "Z".codePointAt(0) ); // 90
3  alert( "z".codePointAt(0) ); // 122
4  alert( "z".codePointAt(0).toString(16) ); // 7a (si necesitamos el valor del cód
```

  - String.fromCodePoint(code).

```
1  alert( String.fromCodePoint(90) ); // Z
2  alert( String.fromCodePoint(0x5a) ); // Z (también podemos usar un valor hexa como
```

  - Comparaciones str.localeCompare(str2):
    - Retorna `1` si `str` es mayor que `str2`.
    - Retorna `-1` si `str` es menor que `str2`.
    - Retorna `0` si son equivalentes.

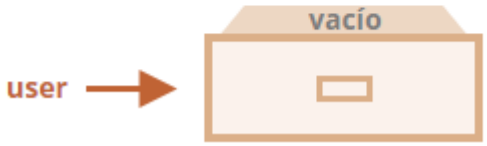
    Por ejemplo:

```
1  alert('Österreich'.localeCompare('Zealand')); // -1
```

Objetos.

- Los objetos son usados para almacenar colecciones de varios datos y entidades más complejas.

```
let user = new Object(); // sintaxis de "constructor de objetos"
let user = {};  // sintaxis de "objeto literal"
```




```
let user = {     // un objeto
  name: "John",  // En la clave "name" se almacena el valor "John"
  age: 30        // En la clave "age" se almacena el valor 30
};
```

Tipos de datos.

- **Array.**
  - Declaración.

```
let arr = new Array();
let arr = [];
```

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits[0] ); // Apple
alert( fruits[1] ); // Orange
alert( fruits[2] ); // Plum
```

```
fruits[3] = 'Lemon'; // ahora ["Apple", "Orange", "Pear", "Lemon"]
```

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits.length ); // 3
```

Tipos de datos.

- **Array.**
  - Almacena cualquier tipo.

```
1  // mezcla de valores
2  let arr = [ 'Apple', { name: 'John' }, true, function() { alert('hello'); } ];
3
4  // obtener el objeto del índice 1 y mostrar su nombre
5  alert( arr[1].name ); // John
6
7  // obtener la función del índice 3 y ejecutarla
8  arr[3](); // hello
```

Tipos de datos.

- **Array como cola.**
  - Métodos pop/push, shift/unshift.

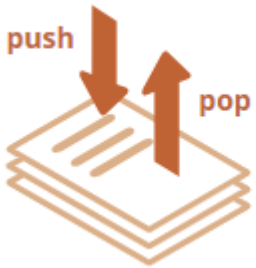



```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.shift() ); // quita Apple y lo muestra en un alert

alert( fruits ); // Orange, Pear
```

- **Array como pila.**




```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.pop() ); // quita "Pear" y lo muestra en un alert

alert( fruits ); // Apple, Orange
```

```
let fruits = ["Apple", "Orange"];

fruits.push("Pear");

alert( fruits ); // Apple, Orange, Pear
```

Tipos de datos.

- **Array.**
  - Bucles.

```
let arr = ["Apple", "Orange", "Pear"];

for (let i = 0; i < arr.length; i++) {
  alert( arr[i] );
}
```

```
let fruits = ["Apple", "Orange", "Plum"];

// itera sobre los elementos del array
for (let fruit of fruits) {
  alert( fruit );
}
```

Tipos de datos.

- **Array.**
  - Longitud de array.

```
let arr = [1, 2, 3, 4, 5];

arr.length = 2; // truncamos a 2 elementos
alert( arr ); // [1, 2]

arr.length = 5; // reponemos la longitud length
alert( arr[3] ); // undefined: el valor no se recupera
```

  - Limpiar un array

    arr.length = 0;

  - Array Multidimensionales.

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

alert( matrix[1][1] ); // 5, el elemento central
```

Tipos de datos.

- **Array. Otros métodos.**
- Splice: inserta, borra y reemplaza elementos

```
let arr = ["Yo", "estudio", "JavaScript"];

arr.splice(1, 1); // desde el índice 1, remover 1 elemento

alert( arr ); // ["Yo", "JavaScript"]
```

```
let arr = ["Yo", "estudio", "JavaScript", "ahora", "mismo"];

// remueve los primeros 3 elementos y los reemplaza con otros
arr.splice(0, 3, "a", "bailar");

alert( arr ) // ahora ["a", "bailar", "ahora", "mismo"]
```

```
let arr = ["Yo", "estudio", "JavaScript", "ahora", "mismo"];

// remueve los 2 primeros elementos
let removed = arr.splice(0, 2);

alert( removed ); // "Yo", "estudio" <-- array de los elementos removidos
```

Tipos de datos.

- **Array. Otros métodos.**
- Slice: devuelve array con copia del orginal.

```
let arr = ["t", "e", "s", "t"];

alert( arr.slice(1, 3) ); // e,s (copia desde 1 hasta 3)

alert( arr.slice(-2) ); // s,t (copia desde -2 hasta el final)
```

- Concat: crea un nuevo array que incluye los valores de otros arrays y elementos adicionales

```
let arr = [1, 2];

// crea un array a partir de: arr y [3,4]
alert( arr.concat([3, 4]) ); // 1,2,3,4

// crea un array a partir de: arr y [3,4] y [5,6]
alert( arr.concat([3, 4], [5, 6]) ); // 1,2,3,4,5,6

// crea un array a partir de: arr y [3,4], luego agrega los valores 5 y 6
alert( arr.concat([3, 4], 5, 6) ); // 1,2,3,4,5,6
```

Tipos de datos.

- **Array. Iteración**
- forEach

```
arr.forEach(function(item, index, array) {
  // ... hacer algo con el elemento
});
```

```
const numeros = [1, 2, 3];

numeros.forEach((elemento, indice, array) => {
  console.log(`Elemento: ${elemento}, Índice: ${indice}`);
});
```

Tipos de datos.

- **Array. Busqueda en array**
- indexOf/lastIndexOf e includes.

```
let arr = [1, 0, false];

alert( arr.indexOf(0) ); // 1
alert( arr.indexOf(false) ); // 2
alert( arr.indexOf(null) ); // -1

alert( arr.includes(1) ); // true
```

```
let fruits = ['Apple', 'Orange', 'Apple']

alert( fruits.indexOf('Apple') ); // 0 (primera "Apple")
alert( fruits.lastIndexOf('Apple') ); // 2 (última "Apple")
```

Tipos de datos.

- **Array. Busqueda en array**
- Find/findIndex y findLastIndex

```
let result = arr.find(function(item, index, array) {
  // si true es devuelto aquí, find devuelve el ítem y la iteración se detiene
  // para el caso en que sea false, devuelve undefined
});
```

```
let users = [
  {id: 1, name: "Celina"},
  {id: 2, name: "David"},
  {id: 3, name: "Federico"}
];

let user = users.find(item => item.id == 1);

alert(user.name); // Celina
```

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"},
  {id: 4, name: "John"}
];

// Encontrar el índice del primer John
alert(users.findIndex(user => user.name == 'John')); // 0

// Encontrar el índice del último John
alert(users.findLastIndex(user => user.name == 'John')); // 3
```

Tipos de datos.

- **Array.**
- Filter: Devuelve array con los elementos encontrados.

```
let results = arr.filter(function(item, index, array) {
  // si devuelve true, el elemento es ingresado al array y la iteración continua
  // si nada es encontrado, devuelve un array vacío
});
```

```
let users = [
  {id: 1, name: "Celina"},
  {id: 2, name: "David"},
  {id: 3, name: "Federico"}
];

// devuelve un array con los dos primeros usuarios
let someUsers = users.filter(item => item.id < 3);

alert(someUsers.length); // 2
```

Tipos de datos.

- **Array. Ordenación**
- Sort:

```
let arr = [ 1, 2, 15 ];

// el método reordena el contenido de arr
arr.sort();

alert( arr );  // 1, 15, 2
```

```
function compareNumeric(a, b) {
  if (a > b) return 1;
  if (a == b) return 0;
  if (a < b) return -1;
}

let arr = [ 1, 2, 15 ];

arr.sort(compareNumeric);

alert(arr);  // 1, 2, 15
```

Tipos de datos.

- **Array.**
- reverse: invierte el orden.

```
let arr = [1, 2, 3, 4, 5];
arr.reverse();

alert( arr ); // 5,4,3,2,1
```

- Split y join. Separan y unen elementos de un array.

```
let nombres = 'Bilbo, Gandalf, Nazgul';

let arr = nombres.split(', ');

for (let name of arr) {
  alert( `Un mensaje para ${name}.` ); // Un mensaje para Bilbo  (y los otros nombr
}
```

```
let str = "test";

alert( str.split('') ); // t,e,s,t
```

```
let arr = ['Bilbo', 'Gandalf', 'Nazgul'];

let str = arr.join(';'); // une el array en una string usando ;

alert( str ); // Bilbo;Gandalf;Nazgul
```

Tipos de datos.

- **Fecha y hora.**
- Creación:

```
let now = new Date();
alert( now ); // muestra en pantalla la fecha y la hora actuales
```

Tipos de datos.

- **Fecha y hora. Metodos**

getFullYear()

Devuelve el año (4 dígitos)

getMonth()

Devuelve el mes, **de 0 a 11**.

```
// fecha actual
let date = new Date();

// la hora en tu zona horaria actual
alert( date.getHours() );

// la hora respecto de la zona horaria UTC+0 (Hora de Londres sin horario de verano)
alert( date.getUTCHours() );
```

getDate()

Devuelve el día del mes desde 1 a 31. Nótese que el nombre del método no es muy intuitivo.

getHours(), getMinutes(), getSeconds(), getMilliseconds()

Devuelve los componentes del horario correspondientes.

getDay()

Devuelve el día de la semana, partiendo de `0` (Domingo) hasta `6` (Sábado). El primer día siempre es el Domingo. Por más que en algunos países no sea así, no se puede modificar.

Tipos de datos.

- **Fecha y hora. Metodos**

getTime()

Devuelve el *timestamp* para una fecha determinada – cantidad de milisegundos transcurridos a partir del 1° de Enero de 1970 UTC+0.

getTimezoneOffset()

Devuelve la diferencia entre UTC y el huso horario de la zona actual, en minutos:

Los siguientes métodos permiten establecer los componentes de fecha y hora:

- `setFullYear(year, [month], [date])`
- `setMonth(month, [date])`
- `setDate(date)`
- `setHours(hour, [min], [sec], [ms])`
- `setMinutes(min, [sec], [ms])`
- `setSeconds(sec, [ms])`
- `setMilliseconds(ms)`
- `setTime(milliseconds)` (Establece la cantidad de segundos transcurridos desde 01.01.1970 GMT+0)

```
let today = new Date();

today.setHours(0);
alert(today); // Sigue siendo el día de hoy, pero con la hora cambiada a 0.

today.setHours(0, 0, 0, 0);
alert(today); // Sigue siendo la fecha de hoy, pero ahora en formato 00:00:00 en
```

A excepción de `setTime()`, todos los demás métodos poseen una variante UTC, por ejemplo: `setUTCHours()`.

Tipos de datos.

- **Fecha y hora. Metodos.**
- Date.parse:

El método Date.parse(str) permite leer una fecha desde un string.

El formato del string debe ser: `YYYY-MM-DDTHH:mm:ss.sssZ`, donde:

- `YYYY-MM-DD` – es la fecha: año-mes-día.
- El carácter `"T"` se usa como delimitador.
- `HH:mm:ss.sss` – es la hora: horas, minutos, segundos y milisegundos.
- El carácter `'Z'` es opcional y especifica la zona horaria, con el formato `+-hh:mm`. Si se incluye únicamente la letra `Z` equivale a UTC+0.

```
1  let date = new Date(Date.parse("2012-01-26T13:51:50.417-07:00"));
2
3  alert(date);
```

Tipos de datos.

- **JSON, toJSON**
- **JSON** (Notación de objeto JavaScript) es un formato general para representar valores y objetos. Se lo describe como el estándar RFC 4627. En un principio fue creado para Javascript, pero varios lenguajes tienen librerías para manejarlo también. Por lo tanto es fácil utilizar JSON para intercambio de información cuando el cliente utiliza JavaScript y el servidor está escrito en Ruby, PHP, Java.

  JavaScript proporciona métodos:

  - JSON.stringify para convertir objetos a JSON.
  - JSON.parse para convertir JSON de vuelta a un objeto.

Tipos de datos.

- **JSON, toJSON**

```
let student = {
  name: 'John',
  age: 30,
  isAdmin: false,
  courses: ['html', 'css', 'js'],
  spouse: null
};

let json = JSON.stringify(student);

alert(typeof json); // ¡obtenemos un string!

alert(json);
/* Objeto JSON-codificado:
{
  "name": "John",
  "age": 30,
  "isAdmin": false,
  "courses": ["html", "css", "js"],
  "spouse": null
}
*/
```

Tipos de datos.

- **JSON, toJSON**
- JSON.parse: Para decodificar un string JSON.

```
let userData = '{ "name": "John", "age": 35, "isAdmin": false, "friends": [0,1,2,3]

let user = JSON.parse(userData);

alert( user.friends[1] ); // 1
```

Clases

- Sintaxis básica.

```
class MyClass {
  // métodos de clase
  constructor() { ... }
  method1() { ... }
  method2() { ... }
  method3() { ... }
  ...
}
```

```
class User {

  constructor(name) {
    this.name = name;
  }

  sayHi() {
    alert(this.name);
  }

}

// Uso:
let user = new User("John");
user.sayHi();
```

- Entonces usamos **new** MyClass() para crear un objeto nuevo con todos los métodos listados.
- El método **constructor**() es llamado automáticamente por new, así podemos inicializar el objeto allí.

Clases

- Sintaxis general de clases

```
class MyClass {
  prop = value; // propiedad

  constructor(...) { // constructor
    // ...
  }

  method(...) {} // método

  get something(...) {} // método getter
  set something(...) {} // método setter

  [Symbol.iterator]() {} // método con nombre calculado (aquí, symbol)
  // ...
}
```

Clases

```
class User {

  constructor(name) {
    // invoca el setter
    this.name = name;
  }

  get name() {
    return this._name;
  }

  set name(value) {
    if (value.length < 4) {
      alert("Nombre demasiado corto.");
      return;
    }
    this._name = value;
  }

}

let user = new User("John");
alert(user.name); // John

user = new User(""); // Nombre demasiado corto.
```

Clases

- Herencia. Palabra clave extends.

```
class Animal {
  constructor(name) {
    this.speed = 0;
    this.name = name;
  }
  run(speed) {
    this.speed = speed;
    alert(`${this.name} corre a una velocidad de ${this.speed}.`);
  }
  stop() {
    this.speed = 0;
    alert(`${this.name} se queda quieto.`);
  }
}

let animal = new Animal("Mi animal");
```

```
class Rabbit extends Animal {
  hide() {
    alert(`¡${this.name} se esconde!`);
  }
}

let rabbit = new Rabbit("Conejo Blanco");

rabbit.run(5); // Conejo Blanco corre a una velocidad de 5.
rabbit.hide(); // ¡Conejo Blanco se esconde!
```

Clases

- Sobreescritura de método.

```
class Rabbit extends Animal {
  stop() {
    // ...esto se usará para rabbit.stop()
    // en lugar de stop() de la clase Animal
  }
}
```

Clases

- Ejemplo.

```
class Animal {

  constructor(name) {
    this.speed = 0;
    this.name = name;
  }

  run(speed) {
    this.speed = speed;
    alert(`${this.name} corre a una velocidad de ${this.speed}.`);
  }

  stop() {
    this.speed = 0;
    alert(`${this.name} se queda quieto.`);
  }

}

class Rabbit extends Animal {
  hide() {
    alert(`¡${this.name} se esconde!`);
  }

  stop() {
    super.stop(); // llama el stop padre
    this.hide(); // y luego hide
  }
}

let rabbit = new Rabbit("Conejo Blanco");

rabbit.run(5); // Conejo Blanco corre a una velocidad de 5.
rabbit.stop(); // Conejo Blanco se queda quieto. ¡Conejo Blanco se esconde!
```

Clases

- Sobreescritura de constructor.

```
class Animal {
  constructor(name) {
    this.speed = 0;
    this.name = name;
  }
  // ...
}

class Rabbit extends Animal {

  constructor(name, earLength) {
    this.speed = 0;
    this.name = name;
    this.earLength = earLength;
  }

  // ...
}

// No funciona!
let rabbit = new Rabbit("Conejo Blanco", 10); // Error:
```

```
class Animal {

  constructor(name) {
    this.speed = 0;
    this.name = name;
  }

  // ...
}

class Rabbit extends Animal {

  constructor(name, earLength) {
    super(name);
    this.earLength = earLength;
  }

  // ...
}

// todo bien ahora
let rabbit = new Rabbit("Conejo Blanco", 10);
alert(rabbit.name); // Conejo Blanco
alert(rabbit.earLength); // 10
```

Clases

- Propiedades y métodos protegidos "_".

```
class CoffeeMachine {
  waterAmount = 0; // la cantidad de agua adentro

  constructor(power) {
    this.power = power;
    alert( `Se creó una máquina de café, poder: ${power}` );
  }

}

// se crea la máquina de café
let coffeeMachine = new CoffeeMachine(100);

// agregar agua
coffeeMachine.waterAmount = 200;
```

```
class CoffeeMachine {
  _waterAmount = 0;

  setWaterAmount(value) {
    if (value < 0) value = 0;
    this._waterAmount = value;
  }

  getWaterAmount() {
    return this._waterAmount;
  }
}

new CoffeeMachine().setWaterAmount(100);
```

Clases

- Propiedades y métodos privados "#".

```
class CoffeeMachine {

  #waterAmount = 0;

  get waterAmount() {
    return this.#waterAmount;
  }

  set waterAmount(value) {
    if (value < 0) value = 0;
    this.#waterAmount = value;
  }
}

let machine = new CoffeeMachine();

machine.waterAmount = 100;
alert(machine.#waterAmount); // Error
```

Manejo de errores

- Try……catch…

```
try {

  // código...

} catch (err) {

  // manipulación de error

}
```

```
try {
  lalala; // error, la variable no está definida!
} catch (err) {
  alert(err.name); // ReferenceError
  alert(err.message); // lalala no está definida!
  alert(err.stack); // ReferenceError: lalala no está definida en (...call stack)

  // También puede mostrar un error como un todo
  // El error se convierte en cadena como "nombre: mensaje"
alert(err); // ReferenceError: lalala no está definido
}
```

```
try {

  alert('Inicio de ejecuciones try');  // (1) <--

  lalala; // error, variable no está definida!

  alert('Fin de try (nunca alcanzado)');  // (2)

} catch (err) {

  alert(`¡Un error ha ocurrido!`); // (3) <--

}
```